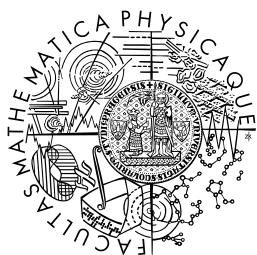


TODOs

1 Include above includes this function	1
--	---



**FACULTY
OF MATHEMATICS
AND PHYSICS**
Charles University

BACHELOR THESIS

Filip Bubák

Graphle: Graph-oriented file management system

Department of Software Engineering

Supervisor of the bachelor thesis: Ing. Pavel Koupil, Ph.D.

Study programme: Programming and Software Development

Prague 2025

The author declares that this bachelor thesis was carried out independently, and only with the cited sources, literature and other professional sources. It is understood that the work relates to the rights and obligations under the Act No. 121/2000 Sb., the Copyright Act, as amended, in particular the fact that the Charles University has the right to conclude a license agreement on the use of this work as a school work pursuant to Section 60 subsection 1 of the Copyright Act.

In date

Author's signature

Acknowledgement Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequaeque doleamus animo, cum corpore dolemus, fieri.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequaeque doleamus animo, cum corpore dolemus, fieri.

Title: Graphle: Graph-oriented file management system

Author: Filip Bubák

Department: Department of Software Engineering

Supervisor: Ing. Pavel Koupil, Ph.D., Katedra softwarového inženýrství

Abstract: Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliquam quaerat voluptatem. Ut enim aequaleam animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At.

Keywords: Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliquam quaerat.

Název práce: Graphle: Grafově orientovaný systém pro správu souborů

Autor: Filip Bubák

Katedra: Katedra softwarového inženýrství

Vedoucí bakalářské práce: Ing. Pavel Koupil, Ph.D., Katedra softwarového inženýrství

Abstrakt: Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliquam quaerat voluptatem. Ut enim aequaleam animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At.

Klíčová slova: Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliquam quaerat.

Contents

1 Introduction	1
2 Analysis	3
2.1 Functional and Qualitative Requirements	3
2.1.1 Functional Requirements	3
2.1.2 Qualitative Requirements	4
2.2 User roles	4
2.2.1 Regular User	4
2.2.2 Power User	4
2.3 Security	4
2.3.1 Accounts	5
2.3.2 File permissions	5
2.4 Use Cases	5
2.4.1 Search for a file	5
2.4.2 Add a relationship	6
2.4.3 Remove a relationship	7
2.4.4 Add a tag	8
2.4.5 Remove a tag	8
2.4.6 Complex DSL query construction	9
2.4.7 Command auto-completion	10
3 Current landscape	12
3.1 Traditional File Managers	12
3.2 Obsidian	12
3.3 TagSpaces	13
3.4 autojump	13
3.5 Comparison	14
4 Design Documentation	15
4.1 Data Model	15
4.2 Architecture	16
4.2.1 Components	16
4.2.2 Communication Interfaces	17
4.2.3 Background Maintenance and Lazy Loading	17
4.3 Module Decomposition	18
4.3.1 System Context	18
4.3.2 Containers	18
4.3.3 GraphleManager Modules	19
4.3.4 GraphleUI Modules	19
4.4 Used Technologies	20
4.4.1 Programming Language	20
4.4.2 API	21
4.4.3 GUI	21
4.4.4 DSL	21

4.4.5 Relationships Between Files	21
4.4.6 Autocomplete	22
4.5 Mockups	22
4.5.1 File Detail Page mockup	22
4.5.2 Filenames Results mockup	23
4.5.3 Find with Relationship Filter mockup	23
5 Developer Documentation	25
5.1 Backend	25
5.1.1 Package Layout	25
5.1.2 Domain Layer	25
5.1.3 Repository Layer	26
5.1.4 Service Layer	26
5.1.5 Controller Layer	26
5.1.6 Configuration	27
5.2 Frontend	27
5.2.1 Package Layout	27
5.2.2 Configuration	27
5.2.3 State Management	28
5.2.4 Backend Communication	28
5.3 Sequence Diagrams	28
5.3.1 Adding a File with a Tag	28
5.3.2 DSL find Query	29
5.3.3 Autocomplete Keystroke	30
5.3.4 Background Sweeper	31
5.4 API Documentation	32
5.4.1 GraphQL	32
5.4.2 REST	33
5.4.3 WebSocket	34
5.4.4 Authentication	34
5.5 Algorithms employed	34
5.5.1 Autocomplete	34
5.5.2 Garbage Collector	35
5.6 DSL	35
5.7 Discussion of implementation problems	36
5.7.1 Real-time autocomplete	36
5.7.2 Transparent computing	36
5.7.3 Portability	37
5.7.4 User navigation	37
5.7.5 Deployment	37
5.8 Testing	37
5.8.1 Autocomplete latency measurement	38
5.8.2 Continuous Integration	38
6 User Documentation	39
6.1 Hardware and Software Requirements	39

6.2 Installation and Deployment	39
6.3 User Interface	39
6.4 Tutorial	39
7 Conclusion	41
Bibliography	42
Appendix A: Vocabulary	43

Section 1

Introduction

TODO: Include above includes this function

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequae doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequae doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequae doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequae doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequae doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem,

ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos.

Section 2

Analysis

Standard filesystems organize files in a strict hierarchy of folders, forcing users to pick a single location for each file and making it difficult to express relationships that span multiple topics or projects. This section provides the analytical documentation for an application designed to extend existing filesystems with relationships and tags, enabling graph based navigation without replacing or breaking the underlying directory structure. It targets both regular users who benefit from a more expressive GUI file browser, and power users such as developers and system administrators who can automate operations via the provided DSL. The section defines functional and qualitative requirements, including work with semantic and hierarchical connections, the interaction with the system, and its qualities. It explains how the system handles user accounts and their file permissions. The last subsection shows possible interactions via use cases, including creating and deleting semantic information, browsing through the filesystem, and using the DSL.

2.1 Functional and Qualitative Requirements

The requirements were defined based on an analysis of existing tools and their limitations. Existing filesystem tools and productivity applications were examined to identify potential enhancements that can be made to them. The existing solutions will be discussed further in the Current landscape section. No formal user interviews were conducted. The requirements were instead derived from experience with the problem domain. The main goal of this project is to extend a standard filesystem to support faster traversal via interpreting the relationships as an LPG.

2.1.1 Functional Requirements

This list serves as a feature list this application should fulfill in order to be considered complete.

- **F1 Backwards Compatibility** - The application must work with the existing filesystems and can work with hierarchies via connections in the graph
- **F2 Relationships between files** - Files can be related to other files
- **F3 File tags** - Files can be categorized via tags
- **F4 File manipulation** - Files can be created, opened, deleted and moved
- **F5 Folders work as files** - Folders can do everything files can do
- **F6 GUI file browser** - Filesystem can be traversed via a GUI client
- **F7 Custom DSL** - Custom DSL for more advanced operations with the filesystem
- **F8 Filename autocompletion** - The application recommends possible filenames based on recently visited files
- **F9 Lazy updates** - Updates on the filesystem are done dynamically as needed
- **F10 Cross-platform support** - The application must run on macOS and Linux

Windows is explicitly not supported, as NTFS assigns each volume an independent drive letter rather than providing a single root hierarchy. Without a common root, files on different drives cannot be connected in the graph.

2.1.2 Qualitative Requirements

This list serves as a requirement list for how well they should handle the users' wishes.

- **Q1.1 Usability - Interaction modes** - The application provides the users with the option to choose between interacting via DSL or GUI
- **Q1.2 Usability - Theming** - Users can set other themes or use a different GUI client altogether
- **Q1.3 Usability - Remote access** - Users can connect remotely
- **Q2.1 Performance - Autocomplete latency** - The autocomplete responds with available filenames within 250ms, ensured by an in-memory index of recently visited files
- **Q3.1 Reliability - Operation retries** - The application will retry performing failing operations before giving up
- **Q3.2 Reliability - Component isolation** - The failure of auxiliary components, such as an auto-completer or GUI, does not prevent the core from running
- **Q4.1 Extensibility - New commands** - The application can be extended to allow new commands

2.2 User roles

The application targets both regular users and power user. The user is able to choose what functionality they want to utilize, and does not need to adjust to a more powerful way of traversing files all at once.

2.2.1 Regular User

A regular user is anyone who wants to organize their files beyond a plain folder hierarchy. They interact with the application through the GUI, browsing the filesystem, assigning tags to files, and navigating relationships between them. No special technical knowledge is required beyond basic computer literacy. This user benefits from a gradual transition to a more expressive way of organizing and discovering files, as the application layers on top of the existing filesystem without requiring any upfront changes.

2.2.2 Power User

A power user is a technically proficient user, most often a developer or system administrator, who wants to automate or script filesystem operations. They interact with the application primarily through the custom DSL, which allows them to easily perform batch operations and more advanced queries that would require a lot of effort using GUI. This user is familiar with command line tools and basic scripting. This role is the ability to integrate the application into their existing workflows and tooling.

2.3 Security

Since the application operates directly on the user's filesystem and can be accessed remotely, security is of utmost importance. To avoid maintaining application-specific security measures, the application delegates its authentication and access control to the underlying operating system. This section describes how user accounts and file permissions are handled.

2.3.1 Accounts

The system will not have the ability to create custom users, as operating systems themselves already provide this functionality. The user account used is the account of the user who launched the application. Hence, to create a new user, you create a new user account in your operating system. This comes with the benefit that the system itself does not need to concern itself with holding user credentials, which could be a significant security liability if the system was exposed to the internet. Furthermore, regular people often set passwords in a way that allows malicious actors to crack them easily. [1] Instead, users are advised to set up SSH port forwarding [2]. This is the method that many other applications, such as DBeaver [3], use for safer access.

2.3.2 File permissions

The application inherits all the file permissions from the user who launched the system. To restrict a user's access, create a new user account for them and assign the necessary permissions.

2.4 Use Cases

This section outlines the use cases describing how the user interacts with the application. Use cases range from a variety of functionalities, including creating and deleting semantic information, browsing through the filesystem, and using the DSL.

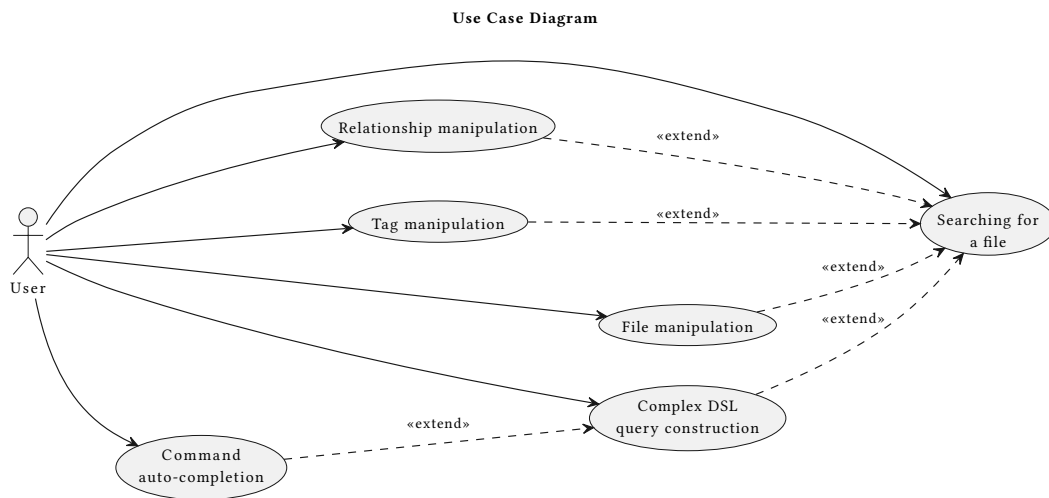


Figure 1: Use case diagram

The Figure 1 shows the overall picture of use cases and how they relate to each other.

2.4.1 Search for a file

Preconditions

- The user has the application opened
- The file browser is connected to a Graphle server
- The user wants to find a file

Flow

1. The user opens the file browser

2. The file browser opens the user-defined home location
3. Application shows the graph of possible neighbors from the current location
4. The user clicks on the neighbor they want to visit
5. If the file was not located then go back to step 3
6. The user clicks on the file
7. The user shows the menu on the file
8. The system shows the menu with different operations available
9. The user selects an operation
10. The system performs the operation

Alternative flow

- 4a) The user uses the integrated DSL for search

Postconditions

- The user accessed the wanted file
- The system performed the desired operation

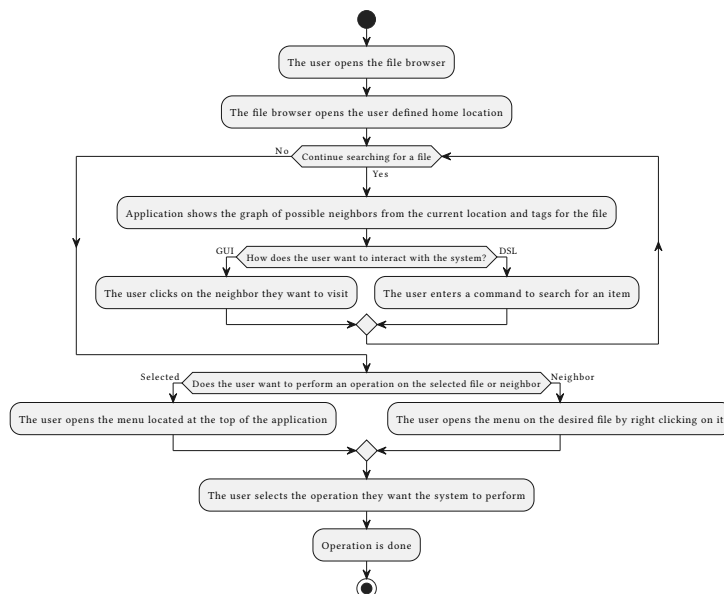


Figure 2: Search for a file activity diagram

2.4.2 Add a relationship

Preconditions

- a) The user has the application opened
- b) The file browser is connected to a Graphle server
- c) The user wants to connect two files with their relationship

Flow

1. The user searches for a file
2. The user selects the operation “Add relationship”
3. The system displays the menu where the user can enter info about the relationship
4. The user enters the information and submits it

Alternative flow

- 1a) The user issues a DSL command
- 4b) The user cancels the operation

Postconditions

- a) The system remembers the new relationship between two entities
- b) The relationship has the name and the value set by the user
- c) The user can traverse the relationship to find the second file straight from the first file

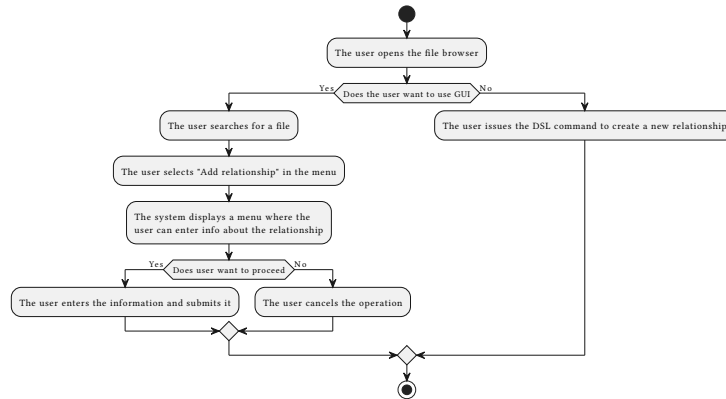


Figure 3: Add a relationship activity diagram

2.4.3 Remove a relationship

Preconditions

- a) The user has the application opened
- b) The file browser is connected to a Graphle server
- c) The user wants to remove a relationship between two files

Flow

1. The user searches for a file
2. The user selects the operation “Remove relationship” on the desired relationship

Alternative flow

- 1a) The user issues a DSL command
- 4b) The user cancels the operation

Postconditions

- a) The system removes the relationship between two entities

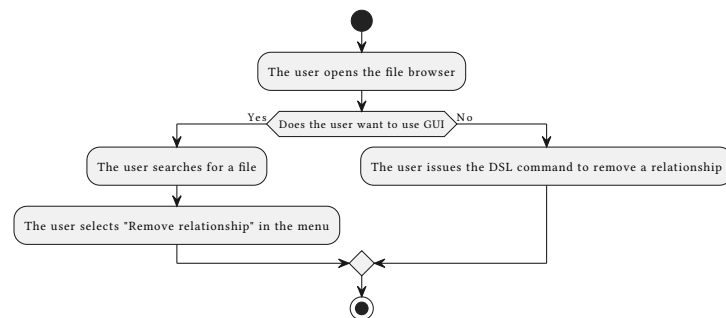


Figure 4: Remove a relationship activity diagram

2.4.4 Add a tag

Preconditions

- a) The user has the application opened
- b) The file browser is connected to a Graphle server
- c) The user wants to tag a file

Flow

1. The user searches for a file
2. The user selects the operation “Add tag”
3. The system displays the menu where the user can enter info about the tag
4. The user enters the information and submits it

Alternative flow

- 1a) The user issues a DSL command
- 4b) The user cancels the operation

Postconditions

- a) The system remembers a new tag
- b) The tag has a name set by the user
- c) The user can find the file by looking at files tagged with such a tag

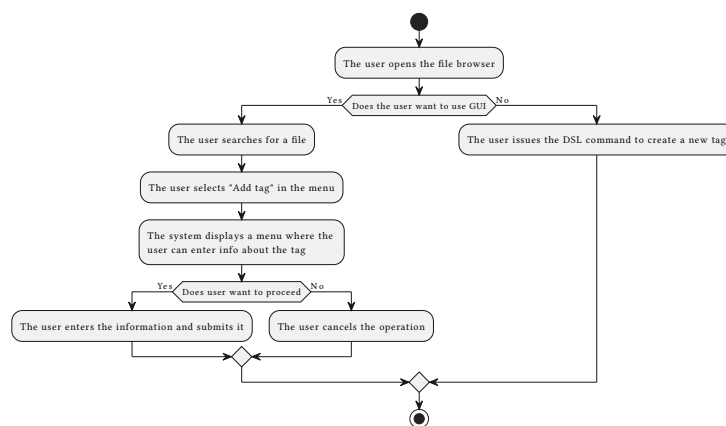


Figure 5: Add a tag activity diagram

2.4.5 Remove a tag

Preconditions

- a) The user has the application opened
- b) The file browser is connected to a Graphle server
- c) The user wants to connect two files with their relationship

Flow

1. The user searches for a file
2. The user selects the operation “Delete” on the desired tag

Alternative flow

- 1a) The user issues a DSL command
- 4b) The user cancels the operation

Postconditions

- a) The system removes the tag

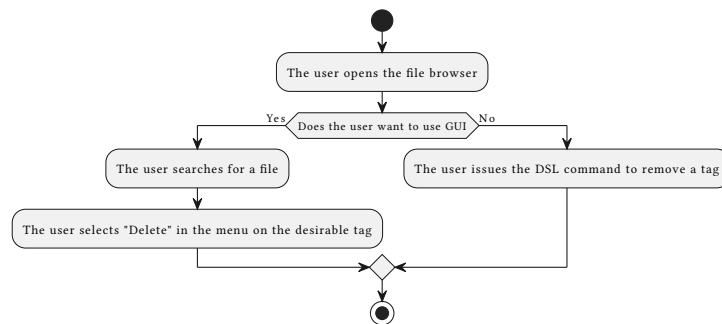


Figure 6: Remove a tag activity diagram

2.4.6 Complex DSL query construction

Preconditions

- The user has the application opened
- The file browser is connected to a Graphle server
- The user wants to perform a complex search using multiple criteria

Flow

1. The user opens the DSL command interface
2. The user starts constructing a query with filters they want to query for (more on that in user tutorial)
3. The user submits the command
4. The system validates the query syntax
5. The system executes the DSL query
6. The system returns the result of the command

Alternative flow

- 4a) If the query syntax is invalid, the system shows an error message and the user returns to step 2

Postconditions

- The user receives a list of objects matching all specified criteria

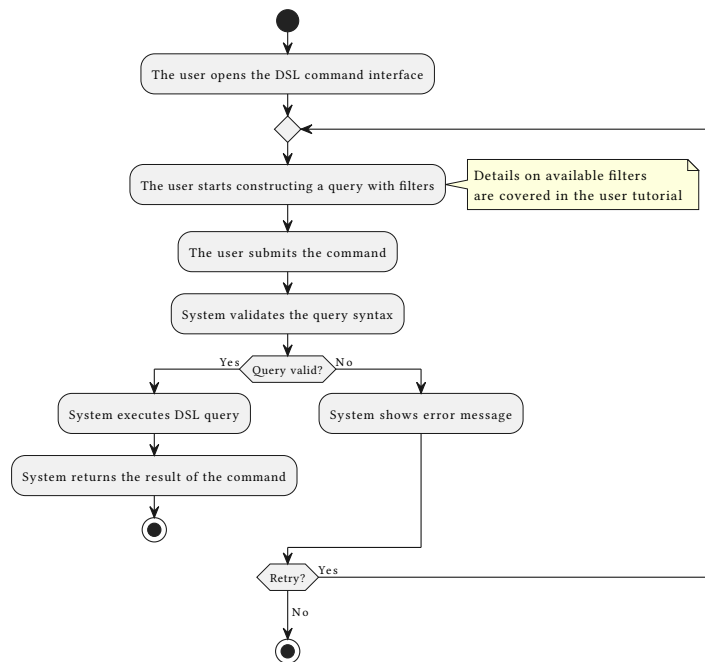


Figure 7: Complex DSL query construction activity diagram

2.4.7 Command auto-completion

Preconditions

- The user has the application opened
- The file browser is connected to a Graphle server
- The DSL command interface is active

Flow

1. The user opens the DSL command interface
2. The user starts typing a command
3. The system analyzes the current input prefix
4. The system queries the auto-completion service
5. The system generates suggestions
6. The system displays a list of suggestions to the user
7. The user selects a suggestion from the list or continues typing
8. If the user selects a suggestion, the system completes the command with the selected text
9. The user continues constructing the command or executes it

Postconditions

- The user has a valid or partially constructed DSL command
- The user saves time by not typing complete paths or commands manually

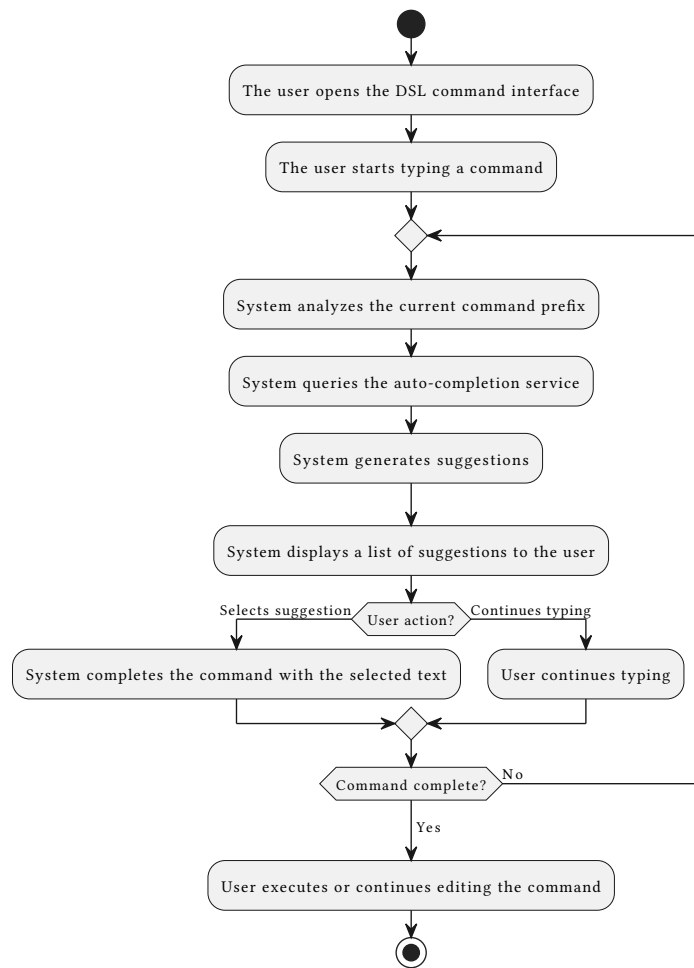


Figure 8: Command auto-completion activity diagram

Section 3

Current landscape

This section examines existing tools that partially address the same user needs as Graphle. The comparison is structured around the functional requirements established in the Functional Requirements section. The criteria used for comparison are those that most directly distinguish the tools. Those include arbitrary relationship support between files, tag support, a query DSL, and remote access, which is relevant to users who manage files on headless servers or NAS devices.

3.1 Traditional File Managers

To understand the problem space, we need to assess the current solutions of file managers. Finder for macOS [4], Nautilus [5] and Dolphin [6] for Linux desktop environments were chosen as representatives of file managers. All three provide a graphical directory browser with file manipulation support. Finder exposes the macOS tagging system, which persists tags as extended filesystem attributes. Nautilus and Dolphin offer no comparable built-in tagging. All three support remote filesystem access. Finder connects via *Connect to Server* (supporting SFTP and SMB), while Nautilus and Dolphin accept `sftp://` URIs directly, allowing users to browse files on remote machines and NAS devices as if they were local.

The closest approximation to relationships available in all three managers is the symbolic link, a pointer that makes a file appear at a location not adjacent to the linked file. Symbolic links require no special application support. However, they are untyped, as they carry no label describing why the two locations are related, and the filesystem keeps no index of what points to a given target, making them non-queryable as a first-class concept. Finding all files symbolically linked to a given target requires a full recursive filesystem scan, which does not scale.

Dolphin exposes an integrated Konsole panel [7] that can be toggled inside the file manager window, giving users a terminal session whose working directory follows the GUI's current folder. The standard shell commands available, such as `cd`, `mv`, `find`, can be used to navigate the filesystem, however they only operate on paths and are not able to navigate based on semantic information. Therefore, it receives a partial rating for DSL part.

3.2 Obsidian

Obsidian [8] is a personal knowledge management application built on top of a local folder of Markdown files. Its central feature is a graph view that visualizes the `wiki` link connections users embed in their notes, establishing explicit, browsable relationships between documents. Because links are stored as plain text, the vault remains readable without the application. Tags are supported via `#hashtag` syntax or YAML front matter.

Despite the graph view, Obsidian is scoped to Markdown files. It cannot express relationships between arbitrary file types such as PDFs, images, or source code files, and folders cannot be tagged or linked as first-class entities. Because Obsidian stores its link index

only for Markdown vaults, non-Markdown files in the same folder are invisible to the graph, hence the *Partial* rating for *Existing FS* in the comparison table below. The Dataview [9] community plugin adds a query language over file metadata, but its queries are still confined to Markdown files. PDFs, images, or a binary cannot appear as a node in a Dataview query.

3.3 TagSpaces

TagSpaces [10] is a cross-platform, open-source file manager that adds a tagging layer directly on top of the existing filesystem. Rather than maintaining a separate database, it encodes tags either in the file name itself (e.g. `report[tax 2024].pdf`) or in a sidecar JSON file placed next to the target, keeping the metadata portable and readable without the application. It supports any file type and includes a built-in viewer for common formats such as images, PDFs, and plain text.

TagSpaces has no concept of directed, typed connections between individual files. Files that share a tag can be gathered through a tag-intersection search, but there is no way to express that *document A cites document B* or that *image X was exported from project Y*. Its query model is limited to tag and filename matching, and there is no composable DSL for queries spanning multiple edges. The application has no remote-access capability.

3.4 autojump

autojump [11] is a shell utility that learns which directories a user visits frequently and allows rapid navigation to them by typing only a partial directory name. Internally it maintains a weighted database of visited paths. The `j <query>` command jumps to the highest-ranked directory whose name contains the query string. This directly addresses quick navigation without requiring the user to remember or type full paths.

autojump is limited to directories and operates only on what the shell has explicitly visited. It carries no concept of relationships between files or directories, no tagging, and no means of querying the filesystem beyond frequency-ranked prefix matching. It is a navigation aid rather than a file management tool, and it provides no graphical interface.

Tool	Relation-ships	Tags	Existing FS	DSL	Auto-complete	Remote Access
Finder	No	Yes	Yes	No	No	Yes
Nautilus	No	No	Yes	No	No	Yes
Dolphin	No	No	Yes	Partial	No	Yes
Obsidian	Partial	Yes	Partial	Plugin	No	No
TagSpaces	No	Yes	Yes	No	No	No
autojump	No	No	Yes	No	Yes	No
Graphle	Yes	Yes	Yes	Yes	Yes	Yes

Table 1: Comparison of existing tools against selected requirements.

3.5 Comparison

None of the surveyed tools satisfies the full requirement set. Standard file managers provide no relationship or tagging model beyond OS-level tags on macOS

Obsidian is the closest conceptual match, a graph of linked documents with tags and partial query support. However, it is constrained to Markdown notes and cannot work with arbitrary file types in its graph. TagSpaces adds tagging to any file type but has no relationship model and no query DSL beyond tag and filename matching. autojump demonstrates that traversing through links through the whole filesystem is useful for navigation, but it has no file management capabilities, no relationship model, and no GUI.

Graphle addresses the gap left by all three by extending a standard filesystem with relationships, tags, a DSL with filename autocompletion, and remote access.

Section 4

Design Documentation

This section describes how the functional and qualitative requirements defined in the analysis are realized in the design of the system. It covers the data model, the overall architecture and communication between components, the technologies selected for each layer, and the visual design of the GUI client. Each design decision is traced back to requirements from analysis they address.

4.1 Data Model

The application works with a graph-oriented extension of the user's existing filesystem. The filesystem itself remains outside the application data model: it stores file contents, permissions, timestamps, and the directory hierarchy. Graphle stores only the metadata needed for graph navigation and derives the rest from the live filesystem when a query is executed.

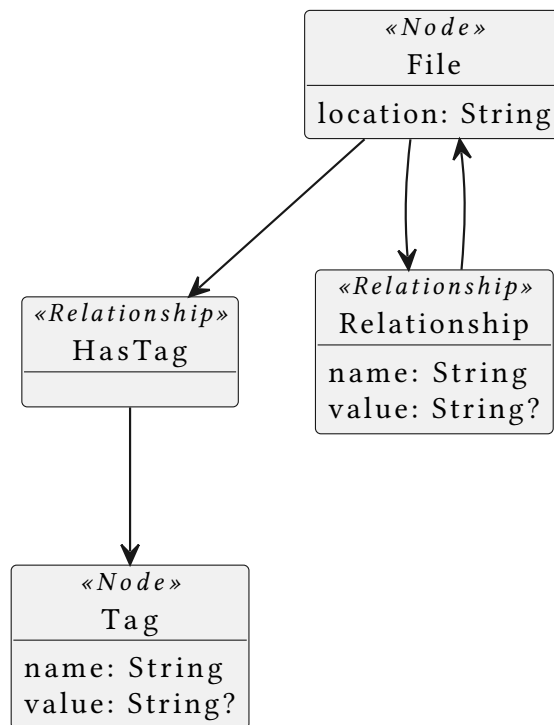


Figure 9: Graphle graph data model

The Figure 9 contains two node types and two edge types. **File** is the central node type. Its `location` attribute is the absolute path on the machine where **GraphleManager** runs and acts as the logical identifier used by the API, DSL, and UI. The database does not duplicate the file's binary content or operating-system metadata. This keeps the model compatible with existing filesystem tools and supports lazy loading (F1, F9).

Tag represents a reusable categorization label. It has a required name and an optional value. The value allows both simple tags, such as `project`, and key-value tags, such as `priority = high`. A file is connected to a tag by a `HasTag` edge. Tags that are no longer connected to any file can be removed without changing the filesystem.

Relationship represents a semantic relationship between two files or folders. It is stored as a directed edge from one File node to another and contains a required name and an optional value. The direction is part of the model, so a relationship from A to B does not imply the reverse relationship. When a user requests a bidirectional relationship, the application represents it as two directed Relationship edges with the same attributes.

The filesystem hierarchy is intentionally not persisted as graph metadata. Parent and descendant relations are computed from the current filesystem state and returned together with stored semantic relationships. This means that moving or deleting a file outside Graphle does not immediately corrupt a stored hierarchy. The next query or background sweep can reconcile the graph with the actual filesystem.

The public API exposes File as an aggregate view containing its location, tags, and visible connections. Additional response types, such as `FilesByTagResponse` and DSL responses, are transport shapes rather than independent domain entities. They exist so clients can display query results and mutation outcomes without introducing new persisted concepts into the data model.

4.2 Architecture

The system is composed of five components: `GraphleManager`, `GraphleUI`, the DSL client, `Neo4j`, and `Valkey`. The application is not purely RESTful. `GraphleManager` exposes three distinct communication interfaces - `GraphQL`, `REST`, and `WebSocket` - each chosen for the specific needs of the client that uses it. The following sections describe how each component and interface addresses the functional and qualitative requirements defined in the analysis.

4.2.1 Components

GraphleManager is the backend service and the single point of contact for all clients. It hosts the DSL interpreter, manages both external datastores, and exposes the three communication interfaces described below. Because `GraphleManager` is a network service, clients can connect to an instance running on a remote machine, giving users access to a remote filesystem (Q1.3). It is implemented on the JVM, which together with `Compose Multiplatform` on the client side ensures the application runs on both macOS and Linux without duplicating the logic (F10). When an auxiliary component such as `Valkey` or a connected GUI becomes unavailable, `GraphleManager` continues to handle all other operations. Failures in optional subsystems are caught and logged without propagating to the core (Q3.2).

GraphleUI is the desktop GUI client built with `Compose Multiplatform` (F6). It communicates with `GraphleManager` exclusively through the public API, which means it can be replaced by any third-party client that speaks the same interfaces (Q1.2). The default client ships with both a light and a dark theme (Q1.2). File and folder manipulation is

exposed through GraphQL mutations for the GUI and through the DSL for the command line client (F4, F5).

DSL client is a command line tool that sends commands to GraphleManager over REST (F7). It is the second user interface available to users alongside the GUI (Q1.1).

Neo4j stores the graph of files, relationships, and tags as a Labeled Property Graph (F2, F3). Only GraphleManager reads from and writes to Neo4j directly, and the clients access the database layer through it. However, the database instance is exposed on a configurable port with a published schema. This allows external applications and independent forks of GraphleManager to query or even extend the dataset directly, fulfilling the extensibility requirement (Q4.1).

Valkey holds the filename autocomplete index in memory as a trie structure (F8). GraphleManager populates and queries this index on behalf of clients. Frequently accessed trie nodes are additionally cached inside GraphleManager using a set of concurrent caches, so hot nodes are served without a network round trip to Valkey. If Valkey is unavailable, autocomplete is silently disabled while all other operations continue (Q3.2).

4.2.2 Communication Interfaces

GraphQL (/graphql) is used by GraphleUI for all file, tag, and connection operations. It allows the client to request exactly the fields it needs, which is important when a query result can contain a large number of files. The schema also gives any future client full transparency over the available data and operations.

REST (/dsl, /download) is used by the DSL client and by GraphleUI for DSL command execution and file downloads. REST was chosen for these endpoints because issuing an HTTP request from the command line is straightforward, whereas constructing a GraphQL query would add unnecessary overhead for CLI use.

WebSocket (/ws) provides a persistent connection for DSL autocomplete. Keeping the connection open avoids establishing a new TCP handshake on each keystroke, which is necessary to meet the latency requirement (Q2.1). GraphleUI reconnects automatically with exponential back-off if the connection is lost. (Q3.1)

4.2.3 Background Maintenance and Lazy Loading

Files, parent directories, and descendants are not indexed eagerly. They are loaded from the filesystem on demand the first time a query requests them (F9). This avoids a potentially unbounded startup scan and keeps the graph populated only with content the user has actually navigated to. Parent and descendant relationships are resolved live by reading the OS filesystem hierarchy rather than being stored in Neo4j, meaning any file already on disk is immediately accessible through Graphle without first being loaded to database (F1).

A background job, **Neo4JSweeper**, runs inside GraphleManager at a configurable interval. It fetches all file locations recorded in Neo4j, checks whether each path still exists on the filesystem, and removes any that have been deleted outside the application along with their orphaned tags and relationships. This keeps the database consistent with the underlying filesystem without requiring the user to manually clean up deleted files.

4.3 Module Decomposition

This section adds a C4 view of the system. The diagrams define the architectural boundaries that follow from the requirements and complement the component-level design. The module names follow the current package structure of GraphleManager and GraphleUI, so the diagrams can be traced back to the implementation.

4.3.1 System Context

At the highest level, Graphle is a desktop-oriented file management system that extends the user's existing filesystem with graph metadata. The operating filesystem remains the source of truth for file existence, hierarchy, and file contents. Graphle stores only semantic metadata in Neo4j and data used for autocomplete in Valkey.

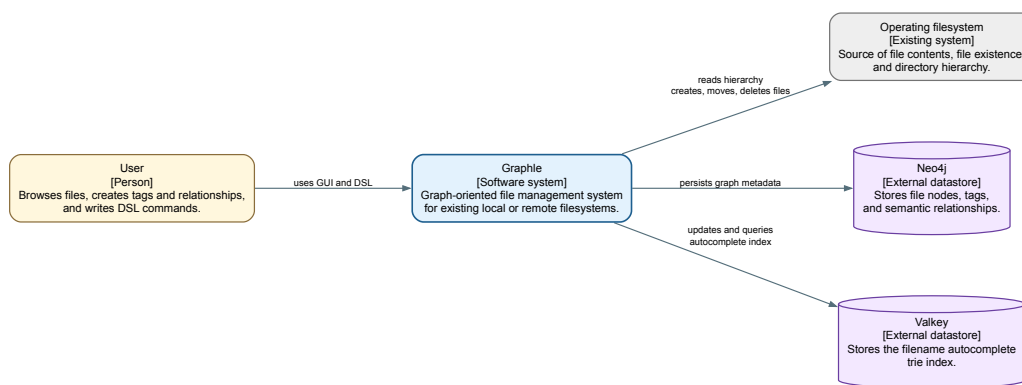


Figure 10: C4 system context diagram

The Figure 10 separates Graphle from the external systems it depends on. This separation is important for backward compatibility (F1), because files can still be manipulated by normal operating-system tools. It is also important for component isolation (Q3.2), because the autocomplete datastore can fail without preventing the core graph and filesystem operations from running.

4.3.2 Containers

The system is split into a desktop GUI container and a backend container. GraphleUI is a Kotlin Compose Multiplatform application. It communicates with GraphleManager through public network interfaces only: GraphQL for file, tag, and relationship operations, REST for DSL execution and file downloads, and WebSocket for command autocomplete. GraphleManager is a Spring Boot service that coordinates filesystem operations, the Neo4j graph, and the Valkey-backed autocomplete index.

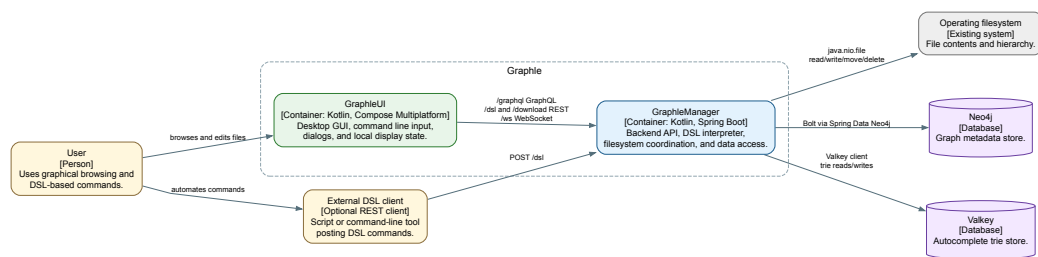


Figure 11: C4 container diagram

The Figure 11 shows that the GUI is not coupled to the database and filesystem directly. This supports remote access (Q1.3) and extensibility (Q4.1), as another client can reuse the same public API instead of having to rely on the current state of backend internals.

4.3.3 GraphleManager Modules

The backend module decomposition follows the source packages in `GraphleManager/src/main/kotlin/com/graphle/graphlemanager`. The API adapters accept client traffic and delegate to service modules. The service modules contain the application logic and decide whether an operation needs Neo4j, the live filesystem, Valkey, or a combination of them.

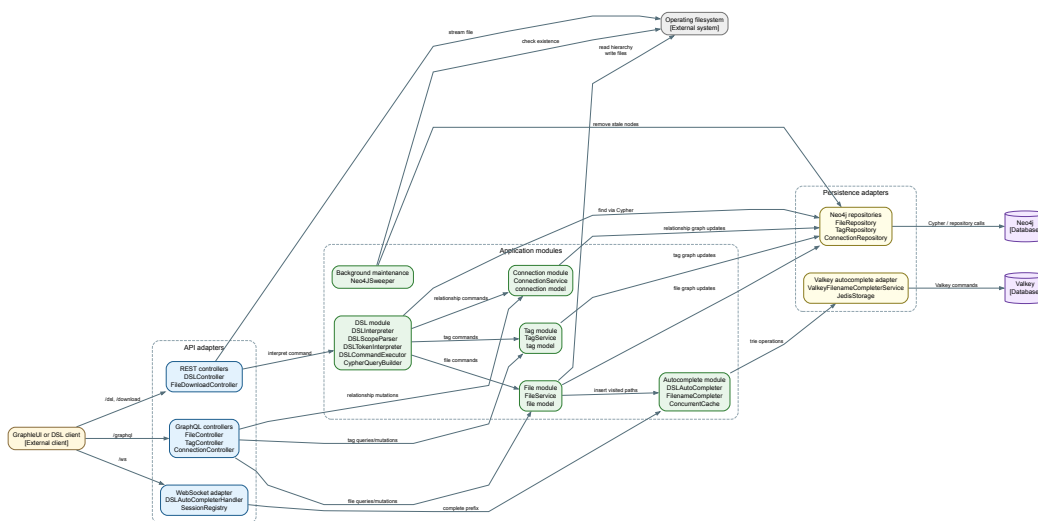


Figure 12: C4 component diagram of GraphleManager

The Figure 12 highlights the main backend boundaries. The file module is the only module that directly writes to the filesystem. The tag and connection modules are graph-specific and persist through Neo4j repositories. The DSL module reuses the same domain services for commands that change state, while complex find queries are translated to Cypher and executed against Neo4j. The autocomplete module owns the Valkey-backed trie and is updated asynchronously from file navigation.

4.3.4 GraphleUI Modules

The frontend module decomposition follows the packages in `GraphleUI/src/main/kotlin/com/graphle`. `App.kt` owns the displayed state and delegates rendering to the

header, dialog, and body modules. All backend communication is kept in transport helpers, which lets the composable UI code work with local models instead of raw GraphQL or HTTP payloads.

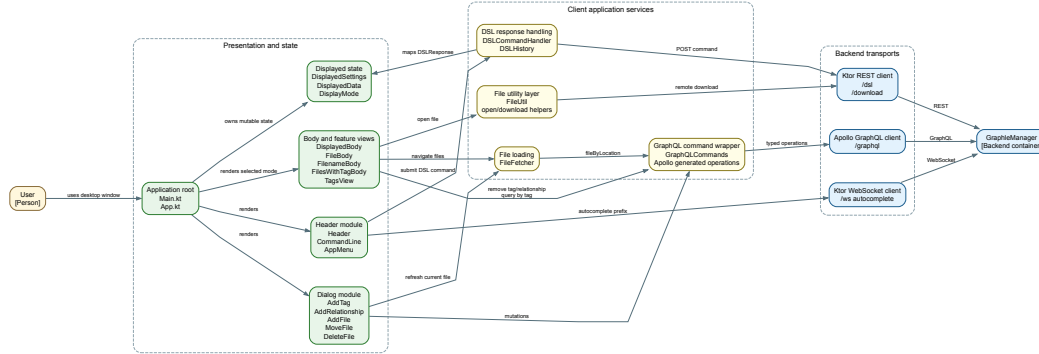


Figure 13: C4 component diagram of GraphleUI

The Figure 13 shows how the GUI keeps its visible content in one shared state model. Actions initiated from the command line, dialogs, or file views are sent to the backend through the appropriate transport. The returned data is converted to `DisplayedSettings`, which App then uses to select and update the currently displayed body. As a result, graphical workflows and DSL-driven workflows update the interface through the same state path.

4.4 Used Technologies

The selection of technologies was guided by three criteria which apply to all components: cross-platform compatibility to ensure the application runs on macOS and Linux without platform-specific defects (requirement F10), ecosystem maturity and community size to lower the barrier for future contributors and reduce the risk of relying on abandoned dependencies, and fit for the specific technical constraints of each component, such as latency requirements or data-model alignment. Where multiple candidates satisfied the first two criteria, the decision was made on technical fit.

4.4.1 Programming Language

The project targets macOS and Linux (requirement F10), so the chosen language must be fully cross-platform. This ensures users do not encounter configuration-specific bugs, reducing the risk of failures that would go undetected during routine developer testing on a single platform.

The language must also be suitable for building large-scale desktop GUI applications and have a sufficiently large community. Since other developers should be able to extend the project with new features, a niche language without an established ecosystem is not suitable. At the time of project inception, the only candidates meeting both constraints were Java [12] and Kotlin [13].

Kotlin was chosen over Java because it supports every Java library while also offering additional language features. This gives developers access to a vast span of existing functionality, allowing them to focus on the core domain rather than reimplementing

general purpose utilities. Kotlin's type system is also more expressive than Java's. Illegal states can be made unrepresentable at compile time more easily, which makes client-server communication less error prone by enforcing a more precise contract.

4.4.2 API

The application is divided into a backend service and two types of clients: a GUI client and a DSL client. Both clients communicate exclusively through the API, which allows either to be swapped out or extended by third-party developers, and enables the backend to serve remote filesystems over a network without architectural changes.

The backend is implemented with Spring Boot [14] and exposes two interfaces. The GUI client communicates via GraphQL [15], which allows it to request exactly the data it needs. This is important since a query result can contain millions of files, and fetching all fields when only a filename is needed would be wasteful. GraphQL also allows the response structure to evolve without introducing a new API version, and provides a schema that gives clients full transparency over the available data and operations. The DSL client communicates via REST, since the DSL is also intended to be used from the command line, where issuing REST calls is simpler than constructing GraphQL queries.

Spring Boot provides a mature ecosystem for building production-grade JVM services, including first-class GraphQL and REST support. Its widespread adoption ensures that contributors with prior JVM experience can onboard with minimal friction.

4.4.3 GUI

The graphical client is built with Compose Multiplatform [16], the dominant GUI framework for Kotlin. Beyond being the most widely adopted option, which eases onboarding for new contributors, Compose Multiplatform supports sharing UI code across desktop, web, and mobile targets from a single codebase. This makes a future port to other platforms a low-cost extension rather than a full rewrite.

4.4.4 DSL

The application provides a domain-specific language (DSL) for advanced filesystem operations. Because autocomplete for the DSL must react to each keystroke, its parser runs inside the backend service and communicates available completions via WebSockets. WebSockets maintain a persistent connection, avoiding the overhead of establishing a new HTTP connection on every keystroke, which is critical for meeting the low-latency autocomplete requirement. gRPC was considered as an alternative, but its additional infrastructure, protobuf schema definitions and code generation, is overkill for a single stream exchanging short strings. There are specific tools for creating DSLs. However, as the auto-completer needs to parse the commands either way, and needs to fetch data from the database, the parser and completer were kept in the same service to avoid serialization between different services, which is important to keep the response time at minimum.

4.4.5 Relationships Between Files

Storing and querying arbitrary relationships between files requires a database whose native data model is a graph. Two graph data models were considered: the Labeled Property Graph (LPG) and the Resource Description Framework (RDF).

RDF was eliminated for two reasons. First, it mandates the use of URIs as node and relationship identifiers. This imposes an unnecessary burden on users who manage a purely local filesystem with no intent of publishing or interlinking it with external datasets. Second, RDF stores are typically triple stores that materialise graph structure only at query time, whereas LPG databases index edges natively, resulting in faster traversal for the relationship-heavy workloads this project targets [17].

Among self-hosted LPG databases, Neo4j [18] is the only option with broad industry adoption [19]. It was therefore selected as the graph store.

4.4.6 Autocomplete

As specified in requirement Q2.1, autocomplete responses must be delivered within the threshold of human reaction time, which is approximately 250 ms [20]. Meeting this constraint requires that candidate filenames be held in memory rather than retrieved from disk on each keystroke.

Valkey [21], a Linux Foundation fork of Redis [22], was selected as the in-memory store. Valkey is reported to be faster and more memory-efficient than Redis in benchmarks [23], and has received corporate backing from Oracle, AWS, and Google. Organisations that now depend on it in production, which reduces the risk of the project being abandoned [24]. Developers familiar with Redis can apply that knowledge directly to Valkey with no additional learning overhead.

To support infix completion while keeping memory consumption reasonable, filenames are indexed using a modified trie. Leaf level path components are stored without their parent segments, enabling efficient prefix search across all components of a filename. Parent path information is preserved as a back reference attached to leaf nodes, so a full path can be reconstructed from any matching suffix without duplicating the entire path at every entry.

4.5 Mockups

This section presents the visual design of the GUI client. Each mockup illustrates a key screen of the application and explains the layout.

4.5.1 File Detail Page mockup

The main page opens at the user's home directory and displays the file detail view. The header contains a hamburger menu and a command line input pre-filled with the current DSL command. Below the header, the body is divided into three sections: URLs (tags whose value is a URL), Tags, and Files. Files are rendered as pills. Parent and descendant connections use arrow icons to indicate direction, while named relationships show their label. This page is displayed for detail on every file, which can be traversed by pressing on the pill or by DSL.

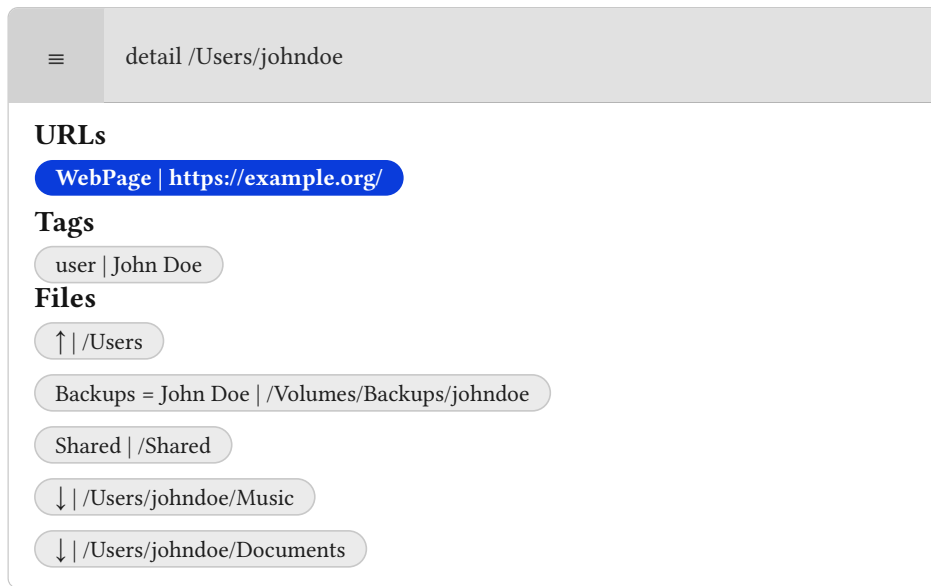


Figure 14: Main page - file detail view

4.5.2 Filenames Results mockup

The filenames results page displays the output of a find DSL command ending with filenames scope. The header shows the executed query, and the body lists the matching files as pills.

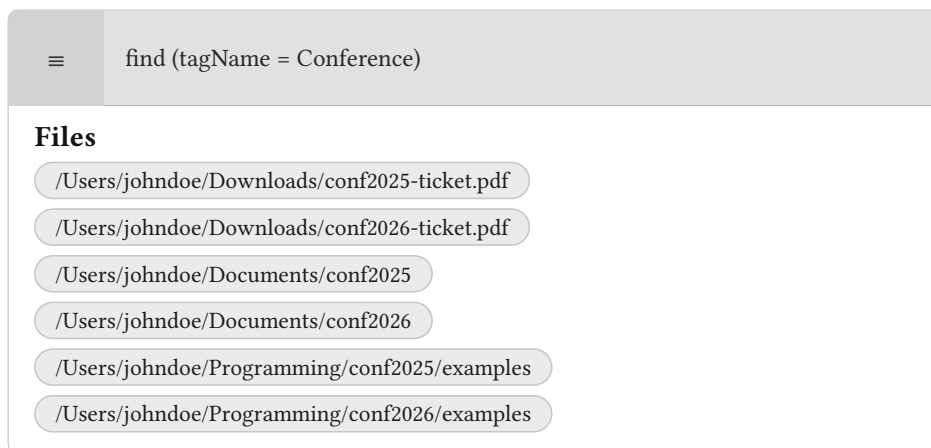


Figure 15: Main page - filenames view

4.5.3 Find with Relationship Filter mockup

When a find command includes a relationship filter, the results page shows matching files as pills. Each pill displays the relationship name and value alongside the file path.

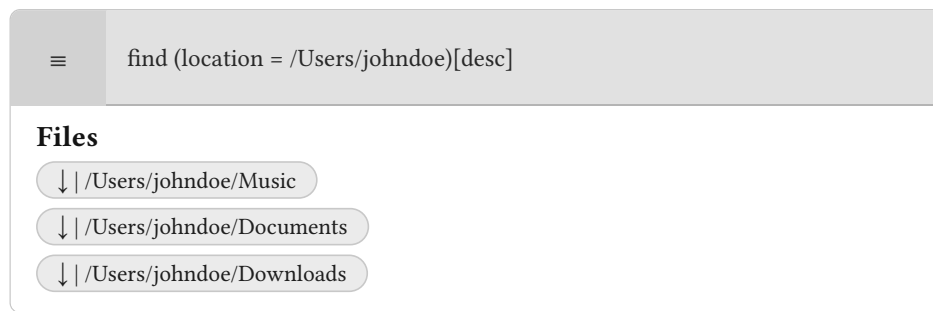


Figure 16: Main page - find results with relationship filter view

Section 5

Developer Documentation

This chapter describes the implementation of Graphle for a developer who wants to work with the source code. It complements the design chapter, which explains why the system is structured the way it is, by focusing on how that structure is implemented. The codebase consists of two applications: `GraphleManager`, a backend written in Kotlin and Spring Boot, and `GraphleUI`, a desktop client written in Compose Multiplatform. These applications can be developed independently, and the only contract the developers need to adhere to is the API communication defined in a subsequent section.

The chapter begins with the package layout of both applications and the concrete classes, repositories, and services that make up each layer, followed by a sequence-diagram walk-through of the most representative end-to-end flows. It then outlines all API endpoints in detail, providing paths, methods, parameters, and example payloads across the GraphQL, REST, and WebSocket interfaces. Next, it describes the algorithms behind the core features, the modified trie used for autocomplete, the background sweeper that keeps the graph consistent with the filesystem, and the multi-stage DSL interpreter pipeline, and discusses the problems that arose during implementation. Finally, the chapter concludes with the testing strategy and the continuous-integration setup that ensures the codebase keeps working as future contributors extend it.

The implementation follows the design chapter without major deviation, and the requirements defined in the analysis are fulfilled by the resulting program.

5.1 Backend

The backend service is `GraphleManager`, a JVM Spring boot application written in Kotlin.

5.1.1 Package Layout

All source lives under `com.graphle.graphlemanager` and is split by domain concern rather than by layer. The root is the Spring entry point `GraphleManagerApplication.kt`. The subpackages are:

- `init/`: Startup configuration.
- `file/`: file entity and the GraphQL/REST endpoints that expose it.
- `tag/`: tag entity and the GraphQL endpoints that expose it.
- `connection/`: relationship entity and the GraphQL endpoints that expose it.
- `dsl/`: the DSL interpreter, autocomplete, and the Valkey-backed trie.
- `sweeper/`: the background cleanup job `Neo4JSweeper`.
- `commons/`: shared utilities.

5.1.2 Domain Layer

`File`, `Tag`, and `Connection` are declared as Spring Data Neo4j entities with UUID identifiers generated by the driver. The `File` node stores the file's absolute path and is linked to its tags and connections. The `Tag` node has an indexed name and an optional value so the same name can take different values on different files. A `Connection` is a named,

directional edge between two File nodes with an optional value and a bidirectional flag. Hierarchical parent and descendant relationships are not persisted and are instead derived from the live filesystem at query time.

Index creation is handled once on startup by `init/Neo4JConfig`, which ensures the mentioned indexes exist.

5.1.3 Repository Layer

Each entity has a `Neo4jRepository` declaring the Cypher queries the rest of the code needs:

- `FileRepository`: `addFileNode`, `removeFileByLocation`, `moveFile`, `getFileLocationsByConnections`.
- `TagRepository`: `tagsByFileLocation`, `addTagToFile`, `removeTag`, `filesByTag`, `removeOrphanTags`.
- `ConnectionRepository`: `neighborsByFileLocation`, `addConnection`, `removeConnection`.

5.1.4 Service Layer

Services sit between the controllers and the repositories and carry all non-trivial logic. `FileService` performs filesystem I/O through `java.nio.file.Files`, reconciles database state with disk state after `addFile/removeFile/moveFile`, and schedules the autocomplete trie asynchronously updates off via `insertFilesToCompleter`. It exposes functions `fileType`, `descendantsOfFile`, and `parentOfFile` that let the controllers compute hierarchical neighbors without going through the database (F1, F9). `TagService` and `ConnectionService` are thin wrappers that delegate to their repositories. Background work is dispatched on a different thread so it does not need to wait and its failure will not affect the rest of the system.

5.1.5 Controller Layer

GraphQL surface is split across three `@Controller` classes, one per entity: `FileController`, `TagController`, and `ConnectionController`. Their methods are tagged with the standard query and mutation annotations, and the nested `File.tags` and `File.connections` fields are fetched lazily, only when the client actually requests them. `FileController.fileByLocation` is the only entry point that crosses domain boundaries: it assembles a single `File` response from the live hierarchical neighbors, the persisted custom connections, and the file's tags, optionally hiding entries marked as hidden by the operating system. Exceptions thrown inside any of these controllers are caught by `@GraphQLExceptionHandler` methods that translate `IOException` and `FileAlreadyExistsException` into readable GraphQL errors used by UI.

Three more endpoints sit outside GraphQL. `DSLController` is a `@RestController` mapped at `/dsl` that hands the request body to the DSL interpreter and returns the resulting response. `FileDownloadController` serves raw file contents over `GET /download?location=...` with a detected MIME type, the GUI uses it whenever the backend is running on a different host. A `WebSocket` endpoint at `/ws` carries autocomplete traffic.

5.1.6 Configuration

The backend reads its settings from `application.properties`, which exposes:

- `server.port`: the HTTP port on which the service listens (default 5824). The GUI's `server.port` must match this value for the transports to reach the backend.
- `spring.graphql.graphiql.enabled`: toggles the built-in GraphiQL playground on the same HTTP port.
- The Neo4j connection: bolt URL and credentials consumed by Spring Data Neo4j.
- The Valkey connection: host and port that the autocomplete layer dials for trie storage.
- `neo4j.sweeper.interval`: how often the background sweeper reconciles the graph with the live filesystem (default one minute).
- `cache.ttl`: the expiration applied to Valkey keys held by the autocomplete trie (default thirty days).

5.2 Frontend

The frontend is `GraphleUI`, a desktop application built with Compose Multiplatform in Kotlin. It targets the JVM and is packaged for supported platforms. Backend access is split between two transports: Apollo GraphQL for all file, tag, and connection operations, and Ktor for the DSL REST call and for the autocomplete WebSocket.

5.2.1 Package Layout

All source lives under `com.graphle` and is split by feature. `Main.kt` opens a single Compose Window around the root App composable, which delegates body rendering to a small dispatcher keyed on the current display mode. The feature packages are:

- `common/`: configuration, singletons, the GraphQL wrapper, a trash helper, the UI state models, and the shared theme and reusable visual primitives.
- `file/`, `tag/`, `fileWithTag/`: one package per main entity, each holding its data model alongside the composables that render it. The file package also collects the fetching, opening, and downloading helpers it needs.
- `dialogs/`: the modal flows for create/delete/move/tag/relationship actions and for surfacing error and invalid-file messages, plus a composite that mounts them inside App.
- `header/`: the search header (command line, theme toggle, menu) and the REST and WebSocket transports it drives.
- `dsl/`: command history and the response dispatcher that turns a DSL reply into a display mode change.

5.2.2 Configuration

The GUI reads a YAML configuration file with a single `server` section that exposes two options:

- `port`: the TCP port on which the backend is listening. Used to build the GraphQL, REST, download, and WebSocket URLs that the transports target.
- `localhost`: a boolean indicating whether the backend runs on the same host as the GUI. When true, the file opening flow skips the HTTP download step and opens the file in place. When false, the GUI always pulls a local copy through `/download` first.

5.2.3 State Management

The UI is driven by a single source of truth held in `App.kt`: a reactive variable `displayedSettings` holding `DisplayedSettings`. `DisplayedSettings` wraps a nullable `DisplayedData` (the currently visible entities: filenames, tags, connections, or a file detail) and a `DisplayMode` enum that selects which body to render. `DisplayedBody.kt` branches on the mode and delegates to the feature specific page type. Child composables never hold the state themselves. App hands them paired getter and setter lambdas (`getDisplayedSettings/setDisplayedSettings`, and analogous ones for the theme), so the source of truth stays unique and any update, whether from a dialog, a menu entry, or a body composable, flows back through the same write path.

5.2.4 Backend Communication

Apollo generates a typed Kotlin client from the server's `schema.graphqls`. `common/GraphQLCommands.kt` exposes generated operations and maps the Apollo DTOs onto the UI-local models.

`DSLRestManager` posts DSL commands to `POST /dsl` on the configured backend and deserialises the `DSLResponse`. `DSLCommandHandler` dispatches on `DSLResponse.type`, chooses the matching `DisplayMode`, and calls an `onEvaluation` callback to update the UI state. `DSLHistory` keeps the last successfully displayed command so the UI can re-submit it after a transient failure (Q3.1).

Real-time autocomplete uses its own connection. `DSLWebSocketManager` keeps a single session against `/ws`, exposes a flow of incoming suggestions, sends outgoing messages, and reconnects with exponential back-off. This keeps the socket hot across keystrokes so latency is dominated by server-side lookup rather than TCP handshake (Q2.1).

5.3 Sequence Diagrams

This section walks through four representative end-to-end flows. The first two are the most common write and read paths a user issues from the GUI and the DSL client. The third describes how the autocomplete loop meets the 250 ms latency budget (Q2.1). The fourth shows the background work that keeps the database consistent with the filesystem (F9). In every diagram, solid arrows represent synchronous calls and dashed arrows represent returned values. The `⌚` glyph marks work performed on the actor's own lifeline (an asynchronous task, a disk write, or an internal computation).

5.3.1 Adding a File with a Tag

The “add file” dialog is triggered from the GUI and a tag is attached to the newly created file. The first round trip creates the file on disk and the `File` node in Neo4j, and, as a side effect, asks the autocomplete service to index the new path. The second round trip merges the tag node and the `HasTag` edge. After both complete, the GUI re-fetches the file detail to display the updated tag set.

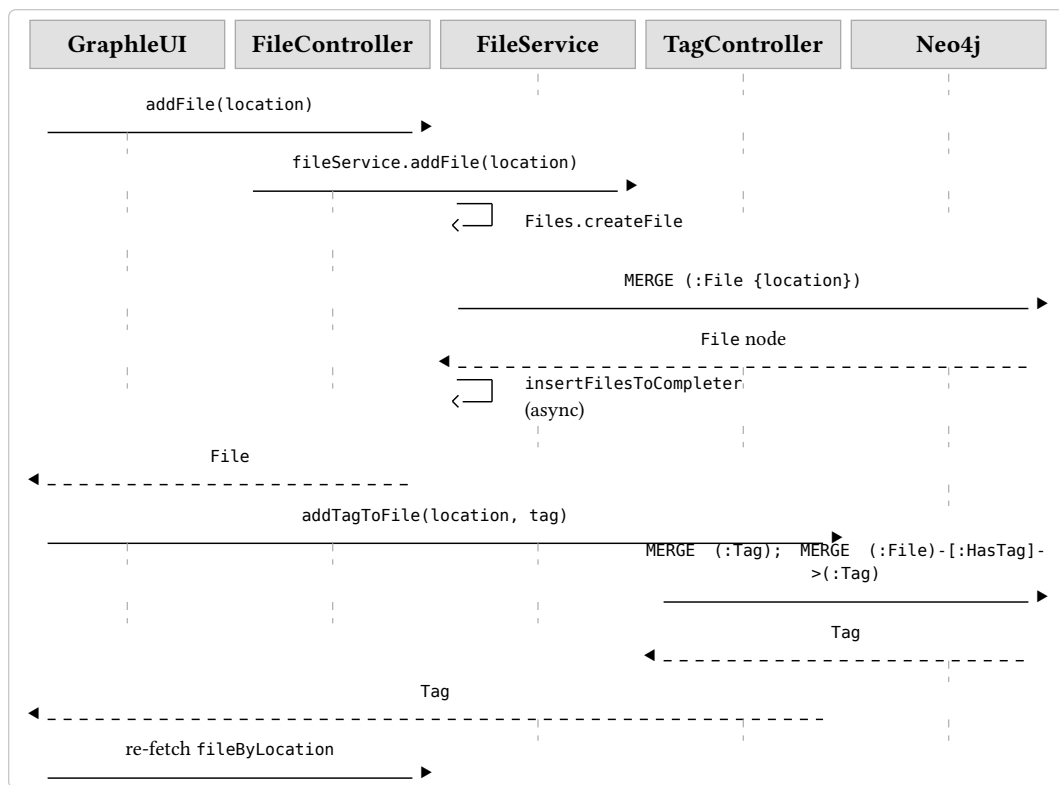


Figure 17: Adding a file and attaching a tag.

5.3.2 DSL find Query

The DSL client posts a command string to /dsl. DSLInterpreter selects the find branch, and the query is split into file- and relationship-level scopes by DSLScopeParser, compiled into Cypher by CypherQueryBuilder, and executed by DSLCommandExecutor against Neo4j. The response type (FILENAMES vs CONNECTIONS) is derived from the last scope of the query so that the client can choose a matching DisplayMode without a second round trip.

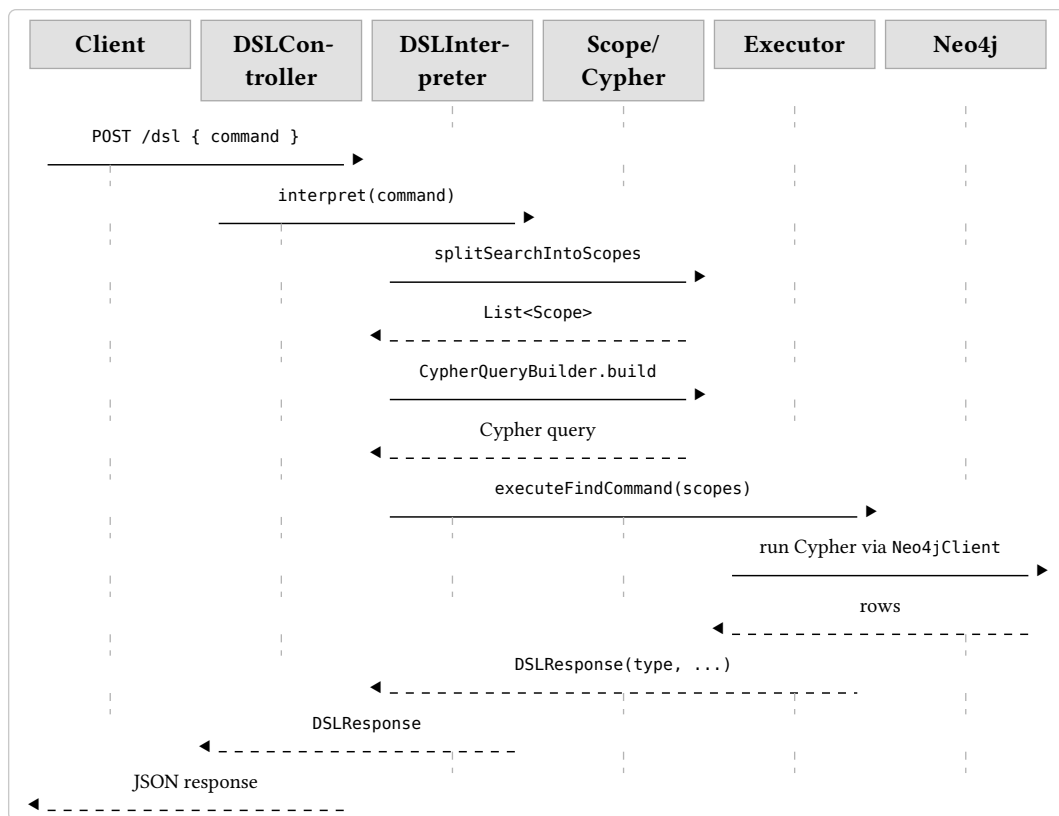


Figure 18: Executing a DSL find query.

5.3.3 Autocomplete Keystroke

Every keystroke in the DSL command line sends the current prefix over a persistent WebSocket. `DSLAutoCompleter` first classifies the command kind from the opening token, then asks `FilenameCompleter` for up to five completions. The filename completer reads its trie primarily from an in-process `ConcurrentCache` and falls back to Valkey only on a miss. Every Valkey access also refreshes the key's TTL so hot entries stay warm for `cache.ttl`.

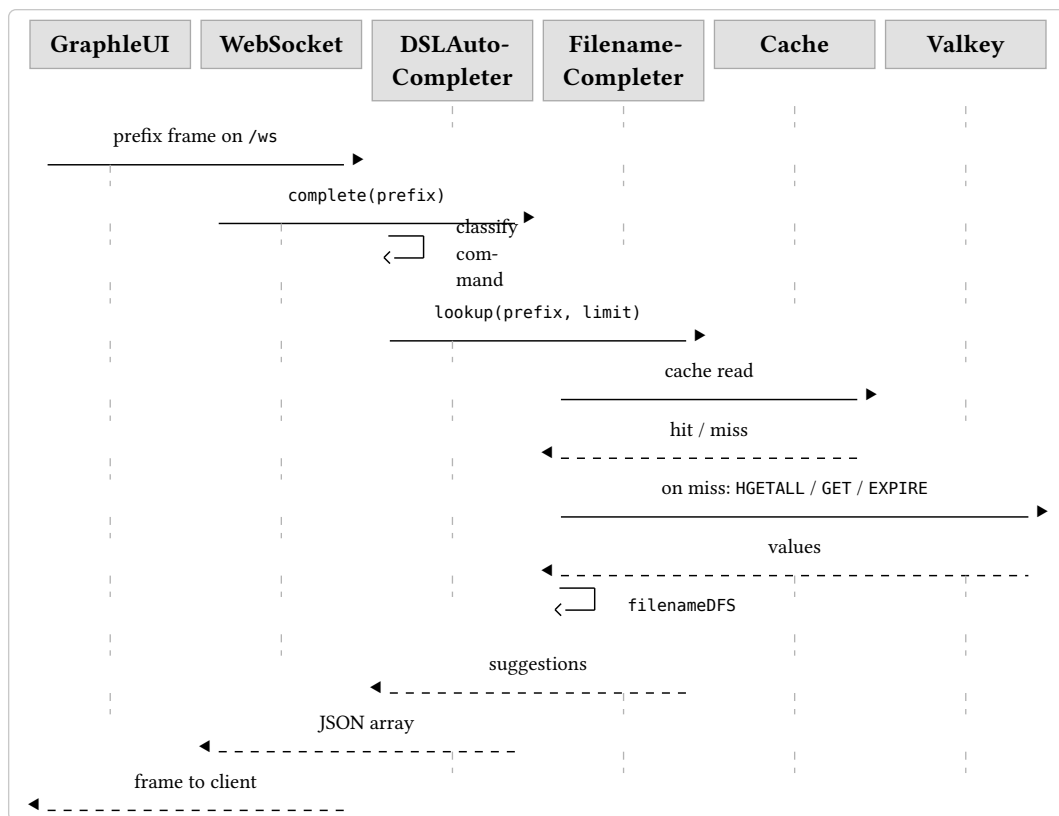


Figure 19: Autocomplete suggestion on a keystroke.

5.3.4 Background Sweeper

Neo4JSweeper runs in its own coroutine and wakes up every `neo4j.sweeper.interval`. It lists all File nodes, tests each for filesystem existence through an injectable predicate, removes the stale ones with `DETACH DELETE`, and then prunes any tag nodes that lost their last incident edge. The sweeper is the only component that reconciles the database with external filesystem changes, which is what allows the rest of the system to treat Neo4j as eventually consistent with the disk.

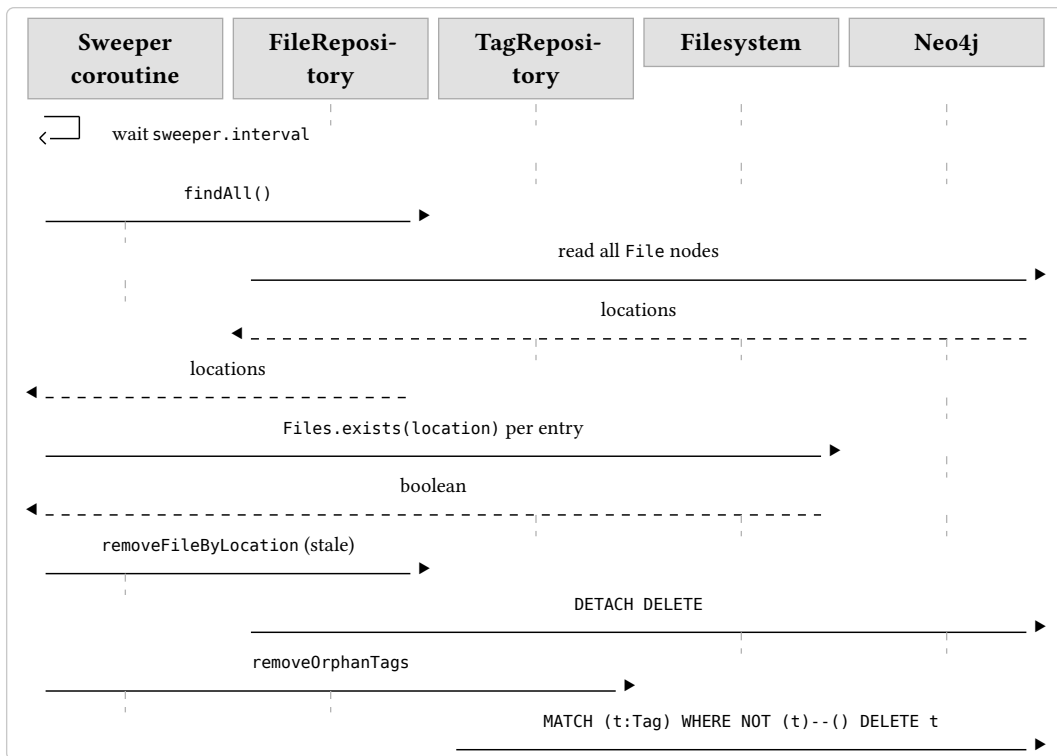


Figure 20: Background sweep of orphaned database state.

5.4 API Documentation

GraphleManager exposes three interfaces on the same HTTP port: GraphQL at `/graphql` for structured CRUD operations, REST under `/dsl` and `/download` for the command-line DSL client and for file transfers, and a WebSocket at `/ws` for real-time autocomplete. All three share the same Spring process and the same security model.

5.4.1 GraphQL

The schema is declared in `GraphleManager/src/main/resources/graphql/schema.graphqls` and is the authoritative source for every type and field. GraphQL is enabled in `application.properties`, so the schema can be explored interactively at `http://<host>:5824/graphql`.

Query	Purpose
<code>fileByLocation(location, showHiddenFiles)</code>	File detail with tags and all live neighbors (parent, descendants, custom connections).
<code>fileType(location)</code>	Returns File or Directory, or null if the path is not present.
<code>tagsByFileLocation(location)</code>	Lists every tag attached to the given file.
<code>filesByTag(tagName)</code>	Reverse lookup from tag to the files that carry it.
<code>filesFromFileByRelationship(fromLocation, relationshipName)</code>	Files reachable from <code>fromLocation</code> via a named relationship (custom or hierarchical).

Query	Purpose
<code>optionsByDslPrefix(dslPrefix, limit)</code>	Autocomplete suggestions for a DSL prefix, offered over GraphQL as an alternative to the WebSocket.
<code>entriesFromDSL(dslCommand, limit)</code>	Executes a DSL command from GraphQL and returns an <code>Entries { entryTypeName, identifiers[] }</code> tuple.

Mutation	Purpose
<code>addFile(location)</code>	Creates the file on disk, adds the File node, schedules an insert into the autocomplete trie.
<code>removeFile(location)</code>	Removes the file from disk and deletes its node along with all its edges from the graph.
<code>moveFile(locationFrom, locationTo)</code>	Moves the file on disk and updates the stored path.
<code>addTagToFile(location, tag)</code>	Creates or updates the tag node and its link to the file.
<code>removeTag(location, tag)</code>	Removes the tag link and garbage-collects the tag node if it becomes orphaned.
<code>addConnection(connection)</code>	Creates a named edge between two File nodes.
<code>removeConnection(connection)</code>	Removes a named edge between two File nodes.

The File type embeds tags: `[Tag!]!` and connections: `[Connection!]!` as nested fields, so a client can request the complete detail in a single round trip. Inputs `TagInput` and `ConnectionInput` mirror their output types. `ConnectionInput.bidirectional` is non-nullable on input so callers must decide whether the new edge should be traversed both ways.

5.4.2 REST

POST `/dsl` accepts `{ "command": "<dsl command>" }` and returns `{ "type": ResponseType, "responseObject": [String] }`. `ResponseType` is one of `FILENAMES`, `CONNECTIONS`, `FILE`, `TAG`, `SUCCESS`, or `ERROR`. The payload shape depends on the type:

- `FILENAMES`: absolute paths.
- `CONNECTIONS`: JSON-serialized `Connection` records.
- `FILE` / `TAG`: JSON-serialized `File` / `TagForFile` records.
- `SUCCESS`: empty list, used by mutating commands such as `addFile`, `addRel`, `addTag`.
- `ERROR`: a single human-readable message.

GET `/download?location=<absolute path>` streams the file contents with a detected `Content-Type`, served by `FileDownloadController`.

5.4.3 WebSocket

`/ws` carries the autocomplete protocol used by the GUI. The client opens a single long-lived session, which is able to reconnect with exponential back-off if it drops, and sends the current DSL prefix on every keystroke. The server answers with a JSON array of suggestions produced by `DSLAutoCompleter`. Keeping the connection persistent avoids a TCP handshake per keystroke, which is what makes the 250 ms budget in Q2.1 achievable.

5.4.4 Authentication

No authentication is performed on any of the three interfaces. `GraphleManager` is intended to run on the operator's own machine or on a trusted LAN host under their control, which matches the remote-access requirement (Q1.3) without exposing the filesystem to untrusted callers. Adding authentication would require a uniform layer across GraphQL, REST, and the WebSocket handshake and is deliberately out of scope.

5.5 Algorithms employed

5.5.1 Autocomplete

Autocomplete is implemented as a trie storing paths. The trie is stored in Valkey, with an in-process cache in front of it to save on round trips to the database. The implementation lives in `dsl/FilenameCompleter.kt` and utilizes a `Storage` abstraction so the trie logic can be tested against a fake or swapped for a different storage backend.

Each trie node is identified by a numeric index, where the root is assigned index 0. Each node stores its parent link back up the trie, the character it represents, a set of back-references to full-path nodes (used for the reverse-lookup described below), and a flag indicating whether it terminates a complete path. A separate top-level counter holds the index of the most recently allocated node so inserts pick a fresh index even across process restarts.

The insertion path implements the “modified trie” described in the architecture chapter. Because many files share long common ancestor paths, storing each full path as an independent sequence of characters would duplicate most nodes. The modified trie avoids this by inserting each path twice: once as the complete absolute path, and once as only its leaf component, with a back-reference pointing from the leaf to the corresponding full-path node. The effect is that a directory prefix shared by many files is stored once and reused rather than repeated for every full path. The parent chain for any leaf can always be reconstructed by following the back-reference to the full-path node and then walking up to the root. Memory therefore grows with the number of unique directory and filename components in the dataset, rather than with the number of files multiplied by their average depth. This matters because the in-process cache stores trie nodes directly. Duplicated prefixes would fill the cache with redundant entries and reduce its effectiveness.

Lookup descends the trie character by character from the root, returning an empty result as soon as a character has no child. From the node reached, `filenameDFS` finds full paths by interleaving three sources. The prefix node itself if it is marked `:full`, the `:parents` set which contains predecessor path indices, and a recursive descent through the children.

Every returned path is checked for existence on disk, so completions pointing at files deleted since the last sweeper run are filtered out before the suggestions are returned.

Five dedicated caches, one per access pattern, sit in front of the storage layer. Each cache entry is refreshed on every access (whether a hit or a miss), so hot nodes stay warm without a separate TTL refresh step. The result is that the first read after startup and all subsequent reads produce the same in-memory view of the data.

To prevent node identifiers from being reused, the database maintains a monotonic last counter and assigns each newly created trie node the next available key. This gives every node a stable identity for its entire lifetime, including across process restarts, which in turn makes the application's in-process cache safer as a cached key can be trusted to refer to the same logical node rather than accidentally pointing to newly inserted trie state after reuse.

5.5.2 Garbage Collector

The sweeper lives in `sweeper/Neo4JSweeper.kt` and keeps the graph consistent with the underlying filesystem (F9). It runs as a single background task that wakes at a configurable interval and executes sweep.

A sweep pass proceeds in two phases. The first phase removes file entries whose recorded locations no longer exist on disk. The second phase removes tag entries that became orphaned as a result of those file deletions.

5.6 DSL

The DSL interpreter is organized as a staged pipeline centered around `DSLInterpreter.interpret`. At the outermost level, the first token identifies the command being invoked. Commands such as file, tag, and relationship updates are parsed into typed inputs and then dispatched to the corresponding application service. The `find` command is structurally richer: rather than mapping directly to a single service operation, it is first parsed, then translated into Cypher, and only afterwards executed against the graph. Keeping this query processing pipeline separate from the dispatch path used by simpler commands makes the language easier to extend. Adding a new command such as a file or tag operation usually requires only a new typed input and a new service branch, while changes to the `find` language remain confined to the parser, query builder, and executor. This reduces coupling between unrelated parts of the DSL and lowers the risk that extending one command family will unintentionally affect another.

Conceptually, a `find` query is an ordered sequence of scopes. Parenthesized scopes describe constraints on file nodes, while bracketed scopes describe constraints on relationships, and the two kinds of scope may be interleaved arbitrarily. This allows a query to express graph traversals incrementally, alternating between conditions on files and conditions on the relationships that connect them. The parser's task is to reconstruct this scoped structure from the input, after which each recovered scope is interpreted as a boolean combination of simple predicates.

Once the query has been decomposed into scopes, `CypherQueryBuilder` translates the DSL representation into Cypher. At this stage, user-facing names and operators are

mapped onto the corresponding graph properties and comparison operators, and small mismatches between user syntax and storage format are normalized. For example, numeric comparisons over tag values are coerced into numeric form even though the underlying values are stored as strings. The result is a query that preserves the user's intent while remaining compatible with the graph schema.

Execution is handled by `DSLCommandExecutor`, which submits the generated Cypher to Neo4j and converts the returned rows into a `DSLResponse`. The shape of the response is determined by the final scope of the query. If the query ends in a relationship scope, the caller receives relationships. If it ends in a file scope, the caller receives filenames. This lets a single surface syntax support several related query patterns without requiring separate command families for each return type.

Autocomplete follows the same structural view of the language. Rather than treating the input buffer as an unstructured string, `DSLAutoCompleter` reconstructs the parse state of the partially written command and infers what kind of token is valid at the cursor position. Filename completion is therefore invoked only where a filename is syntactically expected.

5.7 Discussion of implementation problems

5.7.1 Real-time autocomplete

Requirement Q2.1 caps an autocomplete response at 250 ms, which rules out doing per-keystroke network work that scales with the input. A fresh TCP handshake on every keystroke is avoided by keeping a single WebSocket open on `/ws`. The client transparently reconnects with exponential back-off when the link drops, so an established connection is already in place whenever typing occurs. A Valkey round trip per trie node traversed would in turn make lookup cost linear in the prefix length, which is avoided by fronting each access pattern with an in-process cache and by the modified trie encoding described in the Algorithms section.

The 250 ms target is therefore a guarantee that holds once the caches are hot. Under normal interactive use the relevant trie nodes already sit in cache, so a lookup is bounded by local work plus a single network round trip. It is explicitly **not** a worst-case guarantee for a freshly started session. The first few keystrokes after a cold start may exceed the budget while the in-process layer is repopulated from Valkey.

5.7.2 Transparent computing

Transparent computing in Graphle means a GUI on one machine can drive a filesystem that lives on another (Q1.3). The invariant that holds this together is that all filesystem I/O is performed on the server. The GUI never assumes a path received from the backend is a local path. Paths are meaningful only on the host where `GraphleManager` runs.

Opening a remote file therefore takes a detour. The GUI calls `GET /download?path=...` against `FileDownloadController`, writes the response into a `GraphleDownloads/` directory under the user's home, and hands that local copy to the operating system. When the backend happens to be running on the same host, a `server.localhost` flag in the YAML config skips the download step and opens the file in place.

5.7.3 Portability

The application targets macOS and Linux (F10). Windows is deliberately not supported. See the analysis chapter for the reasoning of not supporting it. Because both the Spring Boot backend and the Compose Desktop frontend are written in Kotlin and compile to JVM bytecode, JDK 21 is the single runtime that makes both components portable. JDK 21 is supported on both MacOS and Linux. Everything else (file existence, directory enumeration, path normalization) delegates to `java.nio.file`, which abstracts the remaining OS differences.

5.7.4 User navigation

The file-detail view is intentionally the central point of the GUI. `FileController.fileByLocation` assembles it in a single round trip by pulling from three independent sources: hierarchical neighbors (`descendantsOfFile`, `parentOfFile`) come from the live filesystem via `FileService`, persisted graph relationships come from `ConnectionController.neighborsByFileLocation`, and tags come from `TagController.tagsByFileLocation`. Before the merged result is returned, every entry is checked with `Files.exists` (and optionally `Files.isHidden`), so a file deleted between the last sweeper run and the current request is silently dropped rather than surfaced to the user.

A subtle consequence is that files which exist on disk but were never explicitly registered with Graphle are still reachable: navigating into a directory shows them under the hierarchical descendants even though they have no `File` node yet (F1). When the user acts on one of those entries, the relevant mutation creates the node as a side effect.

5.7.5 Deployment

`GraphleManager` depends on two external services, Neo4j and Valkey, each with its own lifecycle. `compose.yaml` in the backend directory describes both containers with named volumes (`neo4j_data`, `valkey_data`) so `docker compose up -d` is sufficient to bring up a working development environment. The Gradle `bootRun` task then starts the JVM process itself. The GUI is a separate Gradle project that is either run directly with `./gradlew run` during development or built into a platform-native installer via the Compose Desktop plugin for distribution.

The two services can fail independently, so a Valkey outage while Neo4j remains healthy is an expected operational state. `GraphleManager` treats it as degraded rather than fatal. Autocomplete stops working, but every other request continues normally. (Q3.2)

5.8 Testing

The backend favors integration tests over isolated unit tests. GraphQL resolvers and REST endpoints are exercised through Spring's `MockMvc` against a real Spring context wired up to live Neo4j and Valkey instances, so behavior observed in tests is the same behavior the production process will produce. Pure logic such as DSL token handling, the autocomplete trie, and the concurrent cache is still covered by focused unit tests, but anything that touches persistence is verified end-to-end. Reusable bases (`BaseGraphQLIntegrationTest`, `BaseRestIntegrationTest`) hide the boilerplate of issu-

ing GraphQL operations and HTTP requests, and randomized identifiers keep parallel runs from colliding on shared graph state.

5.8.1 Autocomplete latency measurement

Autocomplete latency was measured manually from the GUI because the perceived responsiveness of the DSL command line depends on the full WebSocket round trip, not only on the pure trie lookup. The client timed each autocomplete request from prefix frame send to matching response frame on an already established /ws connection. The samples therefore exclude connection setup and focus on request handling. This includes Valkey lookup, path reconstruction, filesystem checks, response transfer, and client-side parsing. Test inputs were randomly chosen path prefixes.

Result count	Samples	Minimum	Average	Median	95th percentile	Maximum
0	21	0.9 ms	1.7 ms	1.5 ms	3.2 ms	3.2 ms
1	29	5.2 ms	15.2 ms	15.8 ms	26.5 ms	28.1 ms
5	40	33.7 ms	47.9 ms	46.6 ms	64.1 ms	66.1 ms

Latency mainly increased with the number of valid paths returned. Prefixes with no result completed almost immediately, while the default five suggestions per call were slower because it had to collect, reconstruct, validate, and encode more candidates. Even the slowest measured group stayed well below the 250 ms limit defined by qualitative requirement Q2.1.

5.8.2 Continuous Integration

The GitHub Actions workflow runs on every push and on pull requests targeting main. Neo4j and Valkey are provided as service containers with health checks so the same wiring used locally is reproduced in CI. The automatic action ensures the functionality in main will not regress once new pull requests with new features start coming in.

Section 6

User Documentation

6.1 Hardware and Software Requirements

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat.

6.2 Installation and Deployment

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At.

6.3 User Interface

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

6.4 Tutorial

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et

infinite impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinite impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensiva et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Section 7

Conclusion

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et.

Bibliography

- [1] Katherine Haan, "America's Password Habits: 46% Report Having their Password Stolen Over the Last Year." [Online]. Available: <https://www.forbes.com/advisor/business/software/american-password-habits/>
- [2] Vinayak Baranwal, "SSH Port Forwarding: Local, Remote, and Dynamic Explained." [Online]. Available: <https://www.digitalocean.com/community/tutorials/ssh-port-forwarding>
- [3] "SSH configuration | DBeaver Documentation." [Online]. Available: <https://dbeaver.com/docs/dbeaver/SSH-Configuration/>
- [4] Apple Inc., "Finder – Mac Help." [Online]. Available: <https://support.apple.com/guide/mac-help/finder-mchlp2605/mac>
- [5] The GNOME Project, "GNOME Files (Nautilus)." [Online]. Available: <https://apps.gnome.org/Nautilus/>
- [6] KDE, "Dolphin File Manager." [Online]. Available: <https://apps.kde.org/dolphin/>
- [7] KDE, "Dolphin – Embedded Terminal." [Online]. Available: <https://docs.kde.org/stable5/en/dolphin/dolphin/dolphin-terminal.html>
- [8] Obsidian, "Obsidian - Sharpen your thinking." [Online]. Available: <https://obsidian.md/>
- [9] Michael Brennan, "Dataview – Obsidian Plugin." [Online]. Available: <https://blacksmithgu.github.io/obsidian-dataview/>
- [10] TagSpaces, "Organize your files and folders with tags | TagSpaces." [Online]. Available: <https://www.tagspaces.org/>
- [11] William Ting, "autojump – A cd Command That Learns." [Online]. Available: github.com/wting/autojump
- [12] Oracle, "Java." [Online]. Available: <https://www.java.com/>
- [13] JetBrains, "Kotlin Programming Language." [Online]. Available: <https://kotlinlang.org/>
- [14] VMware, "Spring Boot." [Online]. Available: <https://spring.io/projects/spring-boot>
- [15] The GraphQL Foundation, "GraphQL: A Query Language for Your API." [Online]. Available: <https://graphql.org/>
- [16] JetBrains, "Compose Multiplatform." [Online]. Available: <https://www.jetbrains.com/lp/compose-multiplatform/>
- [17] Neo4j, Inc., "RDF vs. Property Graphs in Knowledge Graphs." [Online]. Available: <https://neo4j.com/blog/knowledge-graph/rdf-vs-property-graphs-knowledge-graphs/>
- [18] Neo4j, Inc., "Neo4j Graph Database." [Online]. Available: <https://neo4j.com/>
- [19] Stack Overflow, "Stack Overflow Developer Survey 2024: Most Popular Technologies – Database." [Online]. Available: <https://survey.stackoverflow.co/2024/technology#most-popular-technologies-database>
- [20] Human Benchmark, "Reaction Time Statistics." [Online]. Available: <https://humanbenchmark.com/tests/reactiontime/statistics>
- [21] The Linux Foundation, "Valkey – An Open Source, In-Memory Data Store." [Online]. Available: <https://valkey.io/>
- [22] Redis Ltd., "Redis." [Online]. Available: <https://redis.io/>
- [23] Redisson, "Valkey vs Redis: Performance Comparison." [Online]. Available: <https://redisson.pro/blog/valkey-vs-redis-comparison.html>
- [24] Edward Targett, "Redis Fork Valkey Joins the Linux Foundation." [Online]. Available: <https://www.thestack.technology/redis-fork-valkey-linux-foundation/>

Appendix A

Vocabulary

API (Application Programming Interface) - A public interface exposed by a software component so other components can call its functionality

Apollo - A GraphQL client library used by GraphleUI to generate and execute typed GraphQL operations

Cache - A faster temporary storage layer used to avoid repeatedly reading the same data from a slower source

Connection - An edge between entities in a database

Relationship - Logical connection between entities describing how they are conceptually related

Neighbor - Entities connected via a direct connection

Tag - Category of an entity

DSL - Domain specific language

Filesystem - The operating system structure that stores files and directories and controls access to them

GraphQL - A query language for APIs that allows clients to request exactly the data they need

HTTP (Hypertext Transfer Protocol) - The request-response protocol used by web APIs and browsers

JSON (JavaScript Object Notation) - A text format for structured data commonly used in API requests and responses

LAN (Local Area Network) - A private network connecting devices in a limited area, such as a home or office

Lazy loading - Loading data only when it is requested instead of eagerly loading everything in advance

LPG (Labeled Property Graph) - A graph data model in which nodes and edges carry both labels and arbitrary key-value properties

Metadata - Data that describes another object, such as tags or relationships describing a file

REST (Representational State Transfer) - An API style that exposes operations through HTTP resources and methods

SFTP (SSH File Transfer Protocol) - A protocol for transferring files over an encrypted SSH connection

SMB (Server Message Block) - A network file sharing protocol commonly used for accessing shared folders

Symbolic link - A filesystem entry that points to another file or directory path

WebSocket - A protocol for a persistent bidirectional connection between a client and a server

Cypher - The declarative query language used by Neo4j to express graph patterns

Trie - A tree data structure where each node represents a character, used for efficient prefix-based lookups